BFMC 2026 – Phase Status Report

Date: March 9, 2026
Prepared by: Team OPTINX
Reporting Period: February 3, 2026 – March 9, 2026
GitHub Repository: https://github.com/mahatosumit/BFMC_2026

1. Overview of Project Status

During the current reporting period, Team OPTINX focused on transitioning from initial system integration toward a stable autonomous driving platform in preparation for the BFMC Qualification Round. A key milestone was the successful integration of the perception pipeline running on a **Raspberry Pi 5** with the low-level actuation system implemented on an **STM32 microcontroller**. This distributed architecture enables real-time interpretation of camera inputs and reliable generation of steering and speed commands while separating high-level perception from deterministic vehicle control.

The primary objective of this phase was to achieve **stable lane keeping and reliable responses to traffic elements**. Extensive testing and parameter tuning were carried out to reduce steering oscillations, stabilize perception outputs, and minimize latency between perception and actuation. These improvements enabled the vehicle to perform **repeatable autonomous runs with consistent trajectory tracking**. To assist development and debugging, a **Tkinter-based monitoring interface** was implemented. The dashboard visualizes perception outputs, telemetry information, and vehicle state in real time. Additionally, a lightweight **topological mapping module** was developed to load GraphML road graphs, compute node-to-node routes, and map the vehicle position along track segments for telemetry monitoring. During experiments, the perception pipeline operates between **19–25 FPS**, depending on scene complexity and lighting conditions, while maintaining stable control loop execution.

2. Current Phase

The current development phase focuses on improving **system robustness and environmental interaction**. As introduced earlier, the system leverages a distributed architecture that separates high-level perception and behavioral logic from deterministic motor control. The Raspberry Pi handles perception processing and decision logic, while the STM32 microcontroller executes steering and motor commands with precise timing.

The perception module combines **classical computer vision and deep learning techniques**. The lane detection pipeline processes frames from the CSI camera and converts them into a **Bird's Eye View representation**. Lane markings are extracted using adaptive thresholding and detected through a sliding-window polynomial fitting method that estimates lane curvature and centerline position. Semantic perception is implemented using a **quantized YOLOv11 model optimized for edge inference** on the Raspberry Pi. The model detects traffic signs, pedestrians, vehicles, and traffic lights. To maintain responsiveness, neural network inference runs **asynchronously in a dedicated thread**, ensuring that perception processing does not interrupt the main control loop. Detected traffic elements are processed through a structured behavioral pipeline consisting of four stages: **PENDING → DETECTING → ACTING → COMPLETED**. This workflow ensures that traffic rules are triggered only when the vehicle reaches the appropriate spatial context and prevents premature execution of behaviors.

Vehicle control is implemented using a **Stanley kinematic controller**, which computes steering commands based on cross-track error and heading deviation. Longitudinal motion is managed by a rule-based decision engine that adjusts vehicle speed depending on road type and environmental conditions. During testing, the vehicle maintains approximately **24 PWM on standard road segments and 26.4 PWM on highway segments**, allowing stable navigation while preserving adequate reaction time for perception-driven decisions.

3. Task Status – Overview

Substantial progress was achieved across perception, control, and system integration during this phase. The lane detection module was refined to improve stability during continuous driving. Temporal smoothing and **Exponential Moving Average filtering** were introduced to stabilize polynomial lane estimates and reduce

steering jitter. As a result, **lane detection accuracy improved from approximately 82% to 93% across experimental runs**.

Controller tuning also significantly improved trajectory stability. Adjustments to the Stanley controller parameters reduced oscillatory corrections, resulting in an **average steering error reduction of approximately 40% during lane tracking**. Perception performance was further optimized through **model quantization and asynchronous neural network inference**, reducing **perception latency from approximately 120 ms to around 55 ms**. These optimizations enabled faster reactions to traffic elements without disrupting the control loop. A fallback mechanism combining **IMU-based dead reckoning and visual odometry** was introduced to maintain trajectory estimation when lane boundaries are temporarily lost. Additional testing validated several autonomous behaviors including **stable lane keeping, pedestrian stopping, parking through CSV-based trajectory playback, and obstacle-aware lane changes**. The system was also evaluated in ramp and tunnel environments, maintaining reliable perception and control despite variations in elevation and lighting.

Reliable communication between system components was ensured through a Python daemon managing serial communication between the Raspberry Pi and STM32 microcontroller. The STM32 executes low-level steering and speed commands, while telemetry data such as steering angle and vehicle velocity are transmitted back to the Raspberry Pi and logged through the BFMC network interface.

Sustainability Considerations

To reduce material consumption during development, several BFMC environmental structures used for testing were constructed primarily using repurposed laboratory materials. Existing boards and surfaces were refurbished and repainted to replicate BFMC elements such as buildings, tunnel sections, and structural slabs, enabling repeated testing without requiring additional construction materials.

4. Blocking Points and Encountered Issues

Several technical challenges were encountered during development and testing. Running both the lane detection pipeline and the YOLO network simultaneously introduced **computational load on the Raspberry Pi 5**, which occasionally caused latency spikes during perception processing. This issue was mitigated through **model quantization and asynchronous inference execution**, significantly reducing computational overhead and preventing blocking of the control loop.

Lighting variations initially affected lane segmentation performance. To improve robustness, camera exposure parameters were calibrated and thresholding methods were adjusted to maintain stable lane extraction under varying illumination conditions. Communication timing between the Raspberry Pi and STM32 microcontroller also required optimization. Differences in processing frequency initially caused **serial buffer overruns and delayed actuation**. This issue was resolved by implementing a lightweight **heartbeat synchronization protocol**, ensuring consistent data exchange between perception and control modules.

Further improvements were achieved by integrating perception-based lane estimation with **IMU feedback**, enabling smoother trajectory correction and consistent adherence to the right-hand lane in accordance with BFMC rules.

5. Next Steps

The immediate focus is to conduct **repeated end-to-end validation runs** to ensure stable lane keeping, reliable detection of traffic elements, and consistent autonomous behavior under different environmental conditions. Following qualification, development will concentrate on improving **behavioral robustness**, refining **intersection handling logic**, and further optimizing perception performance through improved object detection consistency and reduced inference latency.

Future work will also focus on integrating **visual mapping with dead-reckoning techniques** to strengthen localization robustness and support more advanced autonomous navigation capabilities.